Lab 5 Report: Minimax and Alpha-Beta Pruning

Nov 7, 2026 · @Mian Danish

Course CAI601400, Artificial Intelligence Lab, Lab 5 (Week 5, CLO2). Covers Lab Task 1 (hand trace) and Task 2 (leaf counters); Task 3 (reordering) is not part of this report.

Task 1: Hand trace (Minimax)

Both trees have 8 leaves and depth 3: MAX at depth 0, MIN at depth 1, MAX at depth 2. Tree A has 6 scores, padded with two zeros as the manual's code does. Values are computed bottom-up.

Tree A, leaves [3, 5, 2, 9, 0, 7, 0, 0]. Root value 5, so MAX picks the left branch.

Depth	Role	Computation	Result
2	MAX	max(3, 5)	5
2	MAX	max(2, 9)	9
2	MAX	max(0, 7)	7
2	MAX	max(0, 0)	0
1	MIN	min(5, 9)	5
1	MIN	min(7, 0)	0
0	MAX	max(5, 0)	5

Tree B, leaves [3, 5, 2, 9, 3, 5, 2, 9]. Root value 5.

Depth	Role	Computation	Result
2	MAX	max(3, 5)	5
2	MAX	max(2, 9)	9
2	MAX	max(3, 5)	5
2	MAX	max(2, 9)	9
1	MIN	min(5, 9)	5
1	MIN	min(5, 9)	5

Depth	Role	Computation	Result
0	MAX	max(5, 5)	5

Task 2: Leaf-evaluation counters

Both `minimax` and `alphabeta` take an extra `counter` argument, a one-item list. The base case (`depth == target`) adds 1 to `counter[0]` before returning the leaf score, so the counter measures exactly how many leaves each algorithm reads. A list is used so every recursive call updates the same number.

The `trace()` function prints the bottom-up table from Task 1, and `run()` calls it, then runs both algorithms on the same padded tree. It asserts that the hand-trace, Minimax and Alpha-Beta root values are equal, and that Alpha-Beta never evaluates more leaves than Minimax. Alpha-Beta prunes with `if alpha >= beta: break`, which skips the second child of a node once the bounds cross.

Source: `lab5_minimax_alphabeta.py` (submitted alongside this report).

Results

Minimax and Alpha-Beta return the same root value on every tree, matching the hand trace. Alpha-Beta evaluated fewer leaves on two of the three trees (captured program output).

Tree	Leaves	Root value	Minimax leaves	Alpha-Beta leaves	Saved
A (from 5.1)	[3, 5, 2, 9, 0, 7] padded to 8	5	8	8	0
B (from 5.2)	[3, 5, 2, 9, 3, 5, 2, 9]	5	8	6	2
Edge case: all equal	[4, 4, 4, 4, 4, 4, 4, 4]	4	8	5	3

Interpretation and viva notes

Why Tree B saves 2 leaves. After the left subtree, the root has alpha = 5. In the right MIN node, the first MAX child returns 5, so beta becomes 5. Now alpha >= beta (5 >= 5): MIN can no longer give MAX more than 5, so the second child (leaves 2 and 9) is skipped.

Why Tree A saves nothing. In the right MIN node, the first child returns 7 and the second returns 0. The bound only crosses after the last child, so nothing is left to cut. The same value can cost different work depending on leaf order, which is why move ordering matters.

Edge case. With all leaves equal, alpha >= beta fires on ties, so Alpha-Beta still prunes 3 leaves and the value is unchanged. Pruning never alters the Minimax decision, only the work.

Design choices. Counters live in a shared list so recursion needs no globals. The padded tree keeps the index arithmetic (`2*i` , `2*i+1`) valid. Alpha is MAX's best guaranteed value so far, beta is MIN's. The code was checked by asserting equal root values across hand trace, Minimax and Alpha-Beta.

Limitation. The trees are small and fixed, so the saving is tiny. Task 3 (reordering branches) is not covered here.